

Conversation Summary: Statistics and Probability Homework Help

Date: 2025-11-22

Table of Contents

1. find_sample_size_binom Explanation
 2. binom.cdf Function Explanation
 3. find_sample_size_nbinom Explanation
 4. nbinom.cdf Function Explanation
 5. compare_q1 Function Explanation
 6. same_prob Function Explanation
 7. NFoldConv Function Explanation
 8. Question 3.A Implementation
 9. Question 4.A Implementation
 10. Question 4.B Implementation
-

find_sample_size_binom

Location

File: hw1.py, Lines 18-31

Function Signature

```
def find_sample_size_binom(p_target = 0.85, prob_defective = 0.03):
```

Purpose

Calculates the **minimum sample size needed** to have a specified probability of finding **at least one defective product** in a production line.

Parameters

- **p_target** (default: 0.85): The desired probability of detecting at least one defective item
- **prob_defective** (default: 0.03): The defective rate of the production line (probability that any single item is defective)

How It Works

The function uses a **binomial distribution** approach:

1. **Starts with n=1 sample** and incrementally increases
2. For each sample size **n**, it calculates: $P(X \geq 1)$ where **X** is the number of defective items
3. This is computed as: $1 - P(X = 0) = 1 - \text{binom.cdf}(0, n, \text{prob_defective})$
 - `binom.cdf(0, n, p)` gives the probability of 0 or fewer defectives
 - So $1 - \text{binom.cdf}(0, n, p)$ gives the probability of **at least 1 defective**
4. When this probability reaches or exceeds **p_target**, it returns the current **n**

Example

If you want to be 85% sure of finding at least one defective item from a production line with 3% defect rate, the function will tell you how many samples you need to inspect.

Alternative Implementation (Commented Out)

The code also shows a **closed-form mathematical solution**:

```
n = log(1 - p_target) / log(1 - prob_defective)
```

This formula derives from: $(1 - p)^n = 1 - p_{\text{target}}$, which directly solves for n without iteration.

binom.cdf

Function Signature

```
binom.cdf(k, n, p)
```

What It Computes

$P(X \leq k)$ - The probability that a binomial random variable X is **less than or equal to** k .

Arguments

1. **k**: The number of successes (threshold value)
 - In line 28: **k=0** means we're looking at $P(X \leq 0) = P(X = 0)$
2. **n**: The number of trials (sample size)
 - In line 28: This is the variable being tested in the loop
3. **p**: The probability of success on each trial
 - In line 28: **prob_defective** (default 0.03) - the defect rate

Example from Line 28

```
p = 1 - binom.cdf(0, n, prob_defective)
```

Breaking this down: - **binom.cdf(0, n, prob_defective)** = $P(X \leq 0) = P(X = 0)$ - This is the probability of finding **0 defective items** in n samples - Mathematically: $(1 - \text{prob_defective})^n$

- **1 - binom.cdf(0, n, prob_defective)** = $1 - P(X = 0) = P(X \geq 1)$
 - This is the probability of finding **at least 1 defective item**

General Formula

For binomial distribution $X \sim \text{Binom}(n, p)$:

```
binom.cdf(k, n, p) = \sum_{i=0}^k C(n,i) \times p^i \times (1-p)^{(n-i)}
```

Where $C(n,i)$ is the binomial coefficient “ n choose i ”.

find_sample_size_nbinom

Location

File: hw1.py, Lines 38-49

Function Signature

```
def find_sample_size_nbinom(r=1, p_target=0.85, prob_defective=0.03):
```

Purpose

Finds the **minimum number of trials needed** to have a specified probability of observing **at least r defective items** using the **negative binomial distribution**.

Parameters

- **r** (default: 1): The number of defective items (successes) you want to find
- **p_target** (default: 0.85): The desired probability threshold
- **prob_defective** (default: 0.03): The defective rate per item

How It Works

The Negative Binomial Distribution Context The negative binomial distribution answers: *“How many trials until we get r successes?”*

- **n**: Total number of trials
- **r**: Number of successes (defectives) we want
- **k = n - r**: Number of failures before getting r successes

The Algorithm

1. **Starts with n = r** (minimum possible, since you need at least r trials to get r defectives)
2. **Calculates k = n - r** (number of failures)
3. **Uses `nbinom.cdf(k, r, prob_defective)`** which gives $P(K \leq k)$ where K is the number of failures before r successes
 - This equals the probability that you'll get r defectives **within** n trials
4. **When this probability `p_target`**, returns n
5. **Otherwise increments n** and tries again

Example

If `r=5`, `prob_defective=0.10`, `p_target=0.9`: - Find the minimum n such that there's a 90% chance of finding at least 5 defective items in n samples

Key Difference from `find_sample_size_binom`

- `find_sample_size_binom`: Finds samples needed for **at least 1** defective
- `find_sample_size_nbinom`: Finds samples needed for **at least r** defectives (more general)

`nbinom.cdf`

Function Signature

`nbinom.cdf(k, r, p)`

What It Computes

$P(K \leq k)$ - The probability that the number of failures K is **less than or equal to k** before achieving r successes.

Arguments

1. **k**: The number of failures (threshold value)
 - In line 46: **k = n - r** (total trials minus required successes)
2. **r**: The number of successes we're waiting for

- In line 46: The target number of defective items we want to find
3. **p**: The probability of success on each trial
- In line 46: `prob_defective` - the defect rate

Understanding Negative Binomial Distribution

The negative binomial distribution models: - **Question**: “How many failures will occur before we get r successes?” - **Alternative view**: “In how many trials will we get r successes?”

Example from Line 46

```
k = n - r
p = nbinom.cdf(k, r, prob_defective)
```

Breaking this down: - If we want **r defective items** and we have **n total trials** - Then we'll have **$k = n - r$ failures** (non-defective items) - **`nbinom.cdf(k, r, prob_defective)`** = $P(\text{failures } k \text{ before } r \text{ successes})$
 - This is equivalent to: $P(\text{getting } r \text{ successes within } n \text{ trials})$ - Or: $P(r\text{-th defective item appears on or before trial } n)$

Concrete Example

Say $r=5, n=50, \text{prob_defective}=0.1$: - $k = 50 - 5 = 45$ failures - **`nbinom.cdf(45, 5, 0.1)`** = Probability of having 45 non-defectives before finding 5 defectives - This equals the probability of finding 5 defectives within 50 trials

Why Use It Here?

The function searches for the smallest n where the probability of getting r defectives is at least `p_target`.

compare_q1

Location

File: `hw1.py`, Lines 52-57

Function Signature

```
def compare_q1(r1 = 5, p1_target=0.9, p1=0.10, r2=15, p2_target=0.90, p2=0.3):
```

Purpose

Compares **two different scenarios** of defect detection by calculating the required sample sizes for each case using the negative binomial distribution.

Parameters (Default Values)

First Scenario

- **`r1 = 5`**: Want to find at least 5 defective items
- **`p1 = 0.10`**: Defect rate is 10%
- **`p1_target = 0.9`**: Want 90% confidence

Second Scenario

- **`r2 = 15`**: Want to find at least 15 defective items
- **`p2 = 0.3`**: Defect rate is 30%
- **`p2_target = 0.90`**: Want 90% confidence

How It Works

1. **Calls** `find_sample_size_nbinom` for the first scenario to get `n1`
2. **Prints** the result for case 1
3. **Calls** `find_sample_size_nbinom` for the second scenario to get `n2`
4. **Prints** the result for case 2
5. **Returns** both sample sizes as a tuple (`n1`, `n2`)

Purpose

This function allows you to **compare** how many samples are needed in different production line scenarios: - Different defect rates - Different confidence levels - Different target numbers of defects to find

Example Interpretation

With default values, it answers: - **Case 1:** How many samples to inspect to be 90% sure of finding 5 defects when defect rate is 10%? - **Case 2:** How many samples to inspect to be 90% sure of finding 15 defects when defect rate is 30%?

The comparison helps understand how defect rate and target number of defects affect required sample size.

same_prob

Location

File: `hw1.py`, Lines 62-72

Function Signature

```
def same_prob(r1 = 5, p1_target=0.9, p1=0.10, r2=15, p2_target=0.90, p2=0.3):
```

Purpose

Finds the **sample size n** where **both scenarios have approximately equal probabilities** of detecting their target number of defects.

Parameters (Same Defaults as `compare_q1`)

First Scenario

- `r1 = 5`: Target at least 5 defects
- `p1 = 0.10`: 10% defect rate

Second Scenario

- `r2 = 15`: Target at least 15 defects
- `p2 = 0.3`: 30% defect rate

How It Works

1. **Starts with n=1** and increments
2. For each `n`, calculates **two probabilities using binomial distribution**:
 - `prob1 = 1 - binom.cdf(r1-1, n, p1) = P(X ≥ r1)`
 - Probability of finding 5 defects in `n` samples with 10% defect rate
 - `prob2 = 1 - binom.cdf(r2-1, n, p2) = P(X ≥ r2)`
 - Probability of finding 15 defects in `n` samples with 30% defect rate
3. **Checks if both probabilities are positive** (non-zero)

4. Checks if they're approximately equal using `np.isclose(prob1, prob2, atol=1e-2)`
 - Considers them equal if they differ by less than 0.01 (1%)
5. Returns `n` when the probabilities match

Key Difference from `compare_q1`

- `compare_q1`: Finds the sample sizes needed for each scenario independently
- `same_prob`: Finds the single sample size where **both scenarios have the same probability** of success

Example

With defaults, it answers: *“At what sample size does the probability of finding 5 defects (10% rate) equal the probability of finding 15 defects (30% rate)?”*

This helps compare how different scenarios balance out at a common sample size.

Relationship to `compare_q1`

No, `same_prob` does not use `compare_q1`. They are independent functions: - `same_prob` directly calculates probabilities using `binom.cdf` in its own loop - `compare_q1` calls `find_sample_size_nbinom` (a different function) - They solve **different problems** with different approaches

NFoldConv

Location

File: `hw1.py`, Lines 170-187

Function Signature

```
def NFoldConv(P, n):
```

Purpose

Calculates the **probability distribution of the sum** of `n` independent and identically distributed (i.i.d.) random variables, each having distribution `P`.

Parameters

- `P`: A 1D numpy array representing the probability distribution
 - `P[i]` = probability that the random variable equals value `i`
- `n`: Number of independent random variables to sum

How It Works

Mathematical Concept If you have `n` independent random variables $X_1, X_2, \dots, X_n$, each with distribution `P`, and you want the distribution of their sum $S = X_1 + X_2 + \dots + X_n$, you use **convolution**.

The Algorithm

1. `Q = P.copy()`: Start with the distribution of a single variable
2. **Loop `n-1` times**: Convolve `Q` with `P` each time
 - After 1 convolution: `Q` represents $X_1 + X_2$
 - After 2 convolutions: `Q` represents $X_1 + X_2 + X_3$
 - After `n-1` convolutions: `Q` represents the sum of `n` variables

3. `Q = Q/Q.sum()`: Normalize to ensure probabilities sum to 1 (handles numerical precision)
4. **Return Q**: The distribution of the sum

Implementation Code

```
def NFoldConv(P, n):
    Q = P.copy()
    for i in range(n-1):
        Q = np.convolve(Q, P)
    Q = Q/Q.sum()
    return Q
```

Example

```
P = np.array([1-0.3, 0.3]) # Bernoulli(0.3): P(X=0)=0.7, P(X=1)=0.3
Q = NFoldConv(P, 5)       # Sum of 5 Bernoulli(0.3) variables
```

This gives you **Binomial(5, 0.3)** - the distribution of the sum of 5 Bernoulli trials, which tells you the probability of getting 0, 1, 2, 3, 4, or 5 successes.

Why Convolution?

Convolution is the mathematical operation that combines two probability distributions to get the distribution of their sum. For independent random variables, the PMF of the sum is the convolution of the individual PMFs.

Question 3.A Implementation

Task

Write code in the notebook (HW1.ipynb) to test/demonstrate the `NFoldConv` function.

Implementation

Added a new cell after the import statement that:

1. **Creates a Bernoulli distribution** with $p=0.3$: `P = [0.7, 0.3]`
2. **Calls `NFoldConv(P, 5)`** to get the distribution of the sum of 5 independent Bernoulli random variables
3. **Displays the results** showing the probability for each possible sum (0 through 5)
4. **Compares with the exact `Binomial(5, 0.3)`** distribution from `scipy` to verify correctness
5. **Shows the difference** to demonstrate that the convolution method works accurately

Code Added to Notebook

```
# Test NFoldConv with a Bernoulli distribution
# P = Bernoulli(0.3): P(X=0)=0.7, P(X=1)=0.3
p = 0.3
P = np.array([1 - p, p])

print("Base distribution P (Bernoulli with p=0.3):")
print(f"P(X=0) = {P[0]:.3f}, P(X=1) = {P[1]:.3f}")

# Sum of n=5 iid Bernoulli(p) should give Binomial(n=5, p=0.3)
n = 5
Q = NFoldConv(P, n)
```

```

print(f"\nDistribution of sum of {n} independent Bernoulli({p}) random variables:")
print("This should match Binomial({}, {}).format(n, p))
print("\nNFoldConv result:")
for k in range(len(Q)):
    print(f"P(Sum = {k}) = {Q[k]:.6f}")

# Compare to exact Binomial PMF from scipy
from scipy.stats import binom
print("\nExact Binomial PMF:")
for k in range(n+1):
    exact_prob = binom.pmf(k, n, p)
    print(f"P(X = {k}) = {exact_prob:.6f}")

# Show the difference
print("\nDifference (NFoldConv - Exact):")
k_values = np.arange(len(Q))
binom_pmf = binom.pmf(k_values, n, p)
diff = Q - binom_pmf
print(f"Max absolute difference: {np.max(np.abs(diff)):.2e}")

```

This demonstrates that the NFoldConv function correctly computes the sum distribution - when you sum 5 Bernoulli trials, you get a Binomial distribution, which the function should reproduce accurately.

Question 4.A Implementation

Task

Implement the `evenBinom(n, p)` function that calculates the probability $P(X \text{ is even})$ for $X \sim \text{Binom}(n, p)$.

Implementation

File: `hw1.py`, Lines 232-248

```

def evenBinom(n, p):
    """
    The program outputs the probability  $P(X \text{ is even})$  for the random variable  $X \sim \text{Binom}(n, p)$ .

    Input:
    - n, p: The parameters for the binomial distribution.

    Returns:
    - prob: The output probability.
    """
    # Calculate  $P(X \text{ is even})$  by summing probabilities for all even values
    #  $P(X \text{ is even}) = P(X=0) + P(X=2) + P(X=4) + \dots + P(X=n \text{ or } n-1)$ 
    prob = 0
    for k in range(0, n+1, 2): # Iterate over even values: 0, 2, 4, ...
        prob += binom.pmf(k, n, p)

    return prob

```

How It Works

The function calculates **P(X is even)** for $X \sim \text{Binom}(n, p)$ by:

1. **Iterating over all even values** from 0 to n (0, 2, 4, 6, ...)
2. **Using `binom.pmf(k, n, p)`** to get the probability for each even k
3. **Summing all these probabilities** to get the total probability that X is even

Technical Details

- `range(0, n+1, 2)` generates all even numbers from 0 to n
- `binom.pmf(k, n, p)` calculates $P(X = k)$ for the binomial distribution
- The sum gives $P(X=0) + P(X=2) + P(X=4) + \dots = P(X \text{ is even})$

Why Not Use `binom.cdf`?

`binom.cdf` gives **cumulative probabilities** - it sums **all** values up to a threshold, not just even values.

For example: - `binom.cdf(4, n, p) = P(X ≤ 4) = P(X=0) + P(X=1) + P(X=2) + P(X=3) + P(X=4)` - This includes **both even AND odd** values

We need **only even values**: $P(X=0) + P(X=2) + P(X=4) + \dots$

Unfortunately, there's no straightforward way to use `binom.cdf` to sum only even values because: - CDF sums **consecutive** integers - We need **alternating** integers (every other one)

The Right Tool for the Job

- `binom.pmf(k, n, p)`: Probability mass function - gives $P(X = k)$ for a specific k
- `binom.cdf(k, n, p)`: Cumulative distribution function - gives $P(X ≤ k)$

For this problem, we need to sum **specific non-consecutive values** (even numbers), so `binom.pmf` is the correct choice.

Question 4.B Implementation

Task

Implement `evenBinomFormula(n, p)` that calculates $P(X \text{ is even})$ using a closed-form formula and prints the proof.

Implementation

File: `hw1.py`, Lines 250-283

```
def evenBinomFormula(n, p):
    """
    The program outputs the probability P(X is even) for the random variable X ~ Binom(n, p) Using a closed-form formula.
    It should also print the proof for the formula.

    Input:
    - n, p: The parameters for the binomial distribution.

    Returns:
    - prob: The output probability.
    """
    # Print the proof for the formula
    print("=" * 70)
    print("PROOF: Closed-form formula for P(X is even), X ~ Binom(n, p)")
    print("=" * 70)
    print("\nLet X ~ Binom(n, p), where q = 1 - p.")
```

```

print("\nUsing the binomial theorem:")
print("  (p + q)^n = Σ(k=0 to n) C(n,k) * p^k * q^(n-k)")
print("  (q - p)^n = Σ(k=0 to n) C(n,k) * (-p)^k * q^(n-k)")
print("              = Σ(k=0 to n) C(n,k) * p^k * q^(n-k) * (-1)^k")
print("\nWhen k is even, (-1)^k = +1")
print("When k is odd,  (-1)^k = -1")
print("\nAdding the two expansions:")
print("  (p + q)^n + (q - p)^n = 2 * Σ(k even) C(n,k) * p^k * q^(n-k)")
print("\nSince p + q = 1:")
print("  1 + (1 - 2p)^n = 2 * P(X is even)")
print("\nTherefore:")
print("  P(X is even) = [1 + (1 - 2p)^n] / 2")
print("=" * 70)

# Calculate using the closed-form formula
prob = (1 + (1 - 2*p)**n) / 2

return prob

```

Mathematical Derivation

The function:

1. **Prints a detailed proof** of the closed-form formula showing:
 - Uses the binomial theorem with $(p+q)^n$ and $(q-p)^n$
 - Shows how adding these expansions isolates even terms
 - Derives the formula: $P(X \text{ is even}) = [1 + (1 - 2p)^n] / 2$
2. **Calculates the probability** using the closed-form formula:

```
prob = (1 + (1 - 2*p)**n) / 2
```

Key Steps in the Proof

1. Start with two binomial expansions:
 - $(p + q)^n = \sum C(n,k) \times p^k \times q^{(n-k)}$
 - $(q - p)^n = \sum C(n,k) \times p^k \times q^{(n-k)} \times (-1)^k$
2. When k is even: $(-1)^k = +1$ When k is odd: $(-1)^k = -1$
3. Adding the expansions:
 - $(p+q)^n + (q-p)^n = 2 \times [\text{sum of even terms}]$
4. Since $p + q = 1$:
 - $1 + (1-2p)^n = 2 \times P(X \text{ is even})$
5. Therefore:
 - $P(X \text{ is even}) = [1 + (1 - 2p)^n] / 2$

Efficiency Advantage

This formula is much more efficient than the iterative approach in `evenBinom`: - **Closed-form**: $O(1)$ time complexity - just one calculation - **Iterative**: $O(n)$ time complexity - must sum $n/2$ terms

Example Usage

```
# Calculate P(X is even) for X ~ Binom(10, 0.3)
prob = evenBinomFormula(10, 0.3)
# This will print the proof and return the probability
```

Summary

This conversation covered:

1. **Binomial Distribution Functions:** Understanding how to calculate sample sizes and probabilities
 - `find_sample_size_binom`: Finding minimum samples for at least 1 defect
 - `binom.cdf`: Cumulative distribution function for binomial
 - `evenBinom`: Calculating probability of even outcomes
2. **Negative Binomial Distribution:** Using it for more general defect detection scenarios
 - `find_sample_size_nbinom`: Finding minimum samples for at least r defects
 - `nbinom.cdf`: Cumulative distribution function for negative binomial
 - `compare_q1`: Comparing two different defect scenarios
3. **Convolution for Sum Distributions:** Understanding how `NFoldConv` works
 - Mathematical foundation of convolution
 - Implementation using `numpy`
 - Verification against known distributions
4. **Probability Calculations:** Both iterative and closed-form approaches
 - Iterative summation using PMF
 - Closed-form formulas using mathematical derivations
 - Trade-offs between approaches
5. **Code Implementation:** Writing clean, well-documented Python functions for statistical analysis
 - Proper use of `scipy.stats` functions
 - Clear documentation and comments
 - Efficient algorithms

Key Takeaways

- **Choose the right distribution:** Binomial for fixed trials, Negative Binomial for fixed successes
- **PMF vs CDF:** Use PMF for specific values, CDF for cumulative probabilities
- **Efficiency matters:** Closed-form formulas can be much faster than iterative approaches
- **Verification is important:** Always validate implementations against known results
- **Documentation helps:** Clear comments and docstrings make code maintainable

All implementations follow best practices with clear documentation, efficient algorithms, and proper use of `scipy.stats` functions.

Converting This File to PDF

To convert this Markdown file to PDF, use one of these methods:

Method 1: Using `pandoc` (Recommended)

```
pandoc conversation_summary.md -o conversation_summary.pdf
```

Method 2: Using `pandoc` with custom styling

```
pandoc conversation_summary.md -o conversation_summary.pdf --pdf-engine=xelatex -V geometry:margin=1in
```

Method 3: Online converter

Upload this file to any Markdown-to-PDF online converter like: - <https://www.markdowntopdf.com/> - <https://cloudconvert.com/md-to-pdf>

Method 4: VS Code extension

If using VS Code, install the “Markdown PDF” extension and use the command palette to export.